

SMART RECRUIT

Full Technical Stack Document

Recruitment & Job Matching Platform

Prepared from the inspected project archive
Smart-Team-Project-Test.zip

.NET 8
ASP.NET Core

HTML/CSS/JS
Vanilla Frontend

SQL Server
LocalDB + EF Core

JWT
Bearer Security

Document Date: 11 August 2026

1. Executive Technical Overview

Smart Recruit is implemented as a single ASP.NET Core web application that hosts both the backend REST API and the static browser frontend. The backend follows a layered Controller - Service - Repository architecture and uses Entity Framework Core with Microsoft SQL Server LocalDB. The frontend is built entirely with HTML5, CSS3 and vanilla JavaScript and communicates with the backend using the browser Fetch API.

Area	Technology / Choice	Project Evidence
.NET version	.NET 8 target framework (net8.0)	Recruitment-Project.csproj
Frontend	HTML5, CSS3, Vanilla JavaScript (ES6+)	wwwroot/ with 34 HTML, 7 CSS and 10 JS files
Frontend framework	None	No React, Angular, Vue, Bootstrap, Tailwind, jQuery or npm project found
Backend	ASP.NET Core Web API / MVC Controllers	Microsoft.NET.Sdk.Web, Controllers/, Program.cs
Database	Microsoft SQL Server LocalDB	UseSqlServer + (localdb)\MSSQLLocalDB
ORM	Entity Framework Core 8.0.8	Microsoft.EntityFrameworkCore.SqlServer 8.0.8
Authentication	JWT Bearer Tokens	JwtBearer 8.0.8 + custom JwtTokenService
Password security	BCrypt password hashing	BCrypt.Net-Next 4.0.3
API documentation	Swagger UI / OpenAPI	Swashbuckle.AspNetCore 6.6.2
Caching	In-memory application cache	Microsoft.Extensions.Caching.Memory / AddMemoryCache

Important version note

The repository identifies the target framework as net8.0. It does not store the exact installed .NET SDK patch version, so the defensible project statement is “.NET 8”.

1.1 High-Level Architecture

Layer	Technology	Responsibility
Browser UI	HTML + CSS + JavaScript	Pages, forms, dashboards, validation, fetch requests, role-aware navigation
Web host	ASP.NET Core 8	HTTPS redirection, static wwwroot files, middleware pipeline, routing
API layer	ControllerBase REST controllers	HTTP endpoints, authorization and request/response handling
Business layer	C# services + interfaces	Authentication, matching, vacancies, applications, verification, moderation, notifications
Data access	Repositories + EF Core DbContext	Persistence, querying, entity relationships and transactions
Database	SQL Server LocalDB	Relational application data
File storage	Local server file system	CV files and employer verification documents
Cross-cutting	JWT, MemoryCache, middleware	Security, active-account checks, exceptions and caching

2. Frontend Technology / Framework

The frontend does not use a JavaScript framework. It is a traditional multi-page application built with native browser technologies and served directly from ASP.NET Core's wwwroot directory.

Frontend Item	Implementation
Markup	HTML5
Styling	CSS3 with shared and module-specific stylesheets
Programming	Vanilla JavaScript (ES6+)
HTTP client	Browser Fetch API
Authentication state	localStorage for JWT, user data and token expiration
Form uploads	FormData for CV and verification-document uploads
Routing / navigation	Standard HTML pages and JavaScript redirects
Frontend build system	None
Package manager	None / no package.json found
UI framework	None

2.1 Frontend Module Structure

Module	Folder	Primary UI Scope
Module 1 - Core	wwwroot/module1-core	Administrator dashboard, users and user details

Module	Folder	Primary UI Scope
Module 2 - Job Seeker	wwwroot/module2-jobseeker	Dashboard, profile, job search and job details
Module 3 - Employer	wwwroot/module3-employer	Employer profile, verification, vacancies and admin approvals
Module 4 - Applications	wwwroot/module4-applications	Applications, applicants, contact requests and interviews
Module 5 - Trust	wwwroot/module5-trust	Notifications, job reports, moderation and audit
Shared	wwwroot/shared	API client, authentication, guards, UI utilities, common/responsive CSS

2.2 Frontend Integration Pattern

- shared/apiClient.js wraps GET, POST, PUT, PATCH and DELETE requests around fetch().
- JWT tokens are attached as Authorization: Bearer <token> for authenticated API calls.
- shared/auth.js manages login, registration, session storage and role-based home-page routing.
- shared/guards.js validates the authenticated user through GET /api/Auth/me and redirects unauthorized roles.
- JSON requests use application/json; multipart file uploads use FormData.
- The frontend and backend are served by the same ASP.NET Core application, avoiding the need for a separate frontend server.

3. Backend Technology / Framework

The backend is an ASP.NET Core 8 Web API built in C#. It uses attribute-routed MVC controllers, dependency injection, middleware, services and repositories.

Backend Component	Technology / Pattern
Runtime / target	.NET 8 (net8.0)
Framework	ASP.NET Core Web API
API style	REST-style HTTP endpoints returning JSON
Controller model	ControllerBase + attribute routing
Dependency injection	Built-in ASP.NET Core DI container using AddScoped
Serialization	System.Text.Json with ReferenceHandler.IgnoreCycles
Authentication	JWT Bearer authentication
Authorization	Role-based [Authorize] policies/attributes
Middleware	Global exception middleware + active-account middleware
HTTPS	UseHttpsRedirection
Static hosting	UseDefaultFiles + UseStaticFiles
Fallback	MapFallbackToFile("not-found.html")

3.1 Backend NuGet Packages

Package	Version	Purpose
BCrypt.Net-Next	4.0.3	Secure password hashing and verification
Microsoft.AspNetCore.Authentication.JwtBearer	8.0.8	JWT Bearer authentication middleware
Microsoft.EntityFrameworkCore	8.0.8	Core ORM framework
Microsoft.EntityFrameworkCore.Design	8.0.8	Design-time EF Core support
Microsoft.EntityFrameworkCore.SqlServer	8.0.8	SQL Server database provider
Microsoft.EntityFrameworkCore.Tools	8.0.8	EF Core migrations/tooling support
Microsoft.OpenApi	1.6.22	OpenAPI model support
Swashbuckle.AspNetCore	6.6.2	Swagger/OpenAPI generation and Swagger UI

4. Database and Data Persistence

Item	Project Implementation
Database engine	Microsoft SQL Server
Development instance	SQL Server LocalDB: (localdb)\MSSQLLocalDB
Database name	RecruitmentProjectDb
Authentication mode	Trusted_Connection=True (Windows integrated authentication in the configured development connection string)
ORM	Entity Framework Core 8.0.8
Provider	Microsoft.EntityFrameworkCore.SqlServer 8.0.8
Schema management	EF Core migrations (Migrations/ folder)
DbContext	Data/AppDbContext.cs
Data-access pattern	Repository pattern with interfaces and concrete repositories

4.1 Main Entity Sets

AppDbContext exposes the following core entity sets, demonstrating the main relational data model:

Domain	Entity Sets
Identity	Users
Job Seeker	JobSeekerProfiles, Skills, JobSeekerSkills, CvDocuments
Employer	EmployerProfiles, EmployerVerifications, EmployerVerificationDocuments
Vacancies	Vacancies, VacancySkills
Recruitment Flow	JobApplications, ContactRequests, InterviewSchedules
Trust & Safety	Notifications, JobReports, ModerationAuditLogs

4.2 File Storage

Stored File	Storage Approach	Validation
Job Seeker CV	Local server folder: CVStorage/	PDF, DOC, DOCX; maximum 5 MB
Employer verification documents	Local server folder: Storage/EmployerVerificationDocuments/	PDF, JPG, JPEG, PNG; maximum 5 MB

Persistence scope

The inspected project uses SQL Server for structured data and the local application file system for uploaded documents. No cloud object-storage integration was found in the archive.

5. Programming Languages and Data Formats

Language / Format	Where Used
C#	ASP.NET Core controllers, services, repositories, entities, DTOs, middleware, caching and business logic
JavaScript	Frontend API integration, authentication state, UI behavior, validation and module logic
HTML5	Public pages and role-specific application screens
CSS3	Shared responsive design system and module-specific presentation
JSON	API request/response bodies and application configuration files
SQL	Database engine language generated/used through Entity Framework Core rather than a raw-SQL-first application design

Frontend language boundary

The inspected frontend contains plain JavaScript rather than TypeScript and does not rely on a Node/npm build pipeline. Browser-native APIs such as `fetch()`, `FormData` and `localStorage` provide the client-side runtime features.

5.1 Data Exchange Conventions

Concern	Implementation
API payloads	JSON request and response bodies for normal REST operations
File uploads	multipart/form-data through browser <code>FormData</code>
Authentication header	Authorization: Bearer <JWT>
Time values	C# <code>DateTime</code> / JSON date values; frontend formats them in JavaScript
Configuration	JSON files such as <code>appsettings.json</code> and <code>launchSettings.json</code>

6. APIs and Application Services

The project exposes internal REST-style APIs through ASP.NET Core controllers. The API surface is organized around the platform's five functional modules.

API Area	Representative Endpoints	Purpose
Authentication & Admin	POST /api/Auth/login; POST /api/Auth/register/job-seeker; GET /api/Auth/me; GET /api/admin/dashboard	Account creation/login, current user, administrator user management
Job Seeker & Jobs	GET/PUT /api/jobseekers/me/profile; GET /api/jobs; GET /api/jobs/{id}; POST/PUT /api/jobseekers/me/cv	Profile, CV, job discovery and job details
Employer & Vacancies	GET/POST/PUT /api/employers/me/profile; /api/employer-verification; /api/employer/vacancies; /api/admin/vacancies/pending	Employer onboarding, verification, vacancy publication and approvals
Applications & Interviews	POST /api/jobs/{jobId}/applications; /api/employer/contact-requests; /api/jobseekers/me/contact-requests; /api/employer/interviews	Applications, contact requests and interview scheduling
Trust & Moderation	/api/notifications; /api/jobs/{vacancyId}/reports; /api/admin/job-reports; /api/admin/moderation/audit	Notifications, reporting, moderation actions and audit history

6.1 Core Internal Services

Service Group	Examples	Responsibility
Identity	AuthService, JwtTokenService, AdminUserService	Registration/login, token creation, administrator user actions
Job Seeker	JobSeekerService, JobSearchService, MatchingService, CvFileStorageService	Profiles, deterministic matching, search and CV handling
Employer	EmployerService, EmployerVerificationService, VacancyService, VacancyApprovalService	Employer onboarding, document verification and vacancy lifecycle
Recruitment	ApplicationService, ContactRequestService, InterviewScheduleService	Post-application recruitment workflow
Trust & Safety	NotificationService, JobReportService, ModerationService, JobTrustService, ModerationAuditService	Notifications, reporting, trust indicators, enforcement and audit logging
Caching	CacheService	In-memory caching with configurable expiration

6.2 OpenAPI / Swagger

Swagger generation is configured through Swashbuckle. In the Development environment, the application enables Swagger and Swagger UI. The Swagger security scheme is configured for Bearer JWT tokens, allowing authenticated API testing from the documentation UI.

6.3 External Services

External Service	Status in Current Project
Google Meet	Manual HTTPS meeting links are supported in interview data; no Google Meet API client/integration was found.
Email / SMS provider	No direct third-party email or SMS integration found.
Payment gateway	Not used.
Cloud storage	Not used; project stores uploads on the local server file system.
External AI service	Not used; job matching is implemented as an internal deterministic MatchingService.

7. Security, Caching and Cross-Cutting Technologies

Technology	How It Is Used
JWT Bearer	Signed authentication tokens with issuer, audience, expiration and role claims
HMAC SHA-256	JWT signing algorithm used by JwtTokenService
BCrypt	Password hashing; plain-text passwords are not stored by design
Role-based authorization	JobSeeker, Employer and Administrator access enforced at API level
ActiveAccountMiddleware	Checks that authenticated accounts remain active during existing JWT sessions
ExceptionMiddleware	Centralized exception-to-HTTP-response handling
IMemoryCache	Application in-memory caching; default configured expiration is 10 minutes
HTTPS redirection	HTTP requests are redirected to HTTPS where configured
System.Text.Json	JSON serialization with cycle handling

Security documentation note

The project contains JWT configuration in appsettings.json. This technical document intentionally does not reproduce any signing secret or other sensitive configuration value.

8. Other Tools, Libraries and Technologies

Tool / Technology	Use in Project
Git	Repository metadata is included (.git), showing Git-based source control.
Swagger UI	Interactive API discovery/testing in the Development environment.
OpenAPI	Machine-readable API description generated through Swashbuckle.
Entity Framework Core Migrations	Database schema versioning and creation/update workflow.
ASP.NET Core Dependency Injection	Registers repositories and services with scoped lifetimes.
ASP.NET Core Static Files	Hosts the HTML/CSS/JS frontend from wwwroot.
Visual Studio / .NET tooling compatibility	Repository includes a .slnx solution, launchSettings.json and a .http request file; these can be used by modern Visual Studio/.NET tooling.
IIS Express launch profile	A development launch profile is present alongside direct HTTP/HTTPS project profiles.

8.1 Development Endpoints / Profiles

Profile	Configured Address / Behavior
HTTP project profile	http://localhost:5062
HTTPS project profile	https://localhost:7175 and http://localhost:5062
IIS Express	http://localhost:61139 with SSL port 44366
Development launch URL	swagger

9. Technical Stack Summary

Required Document Item	Final Project Answer
.NET Version used	.NET 8 (TargetFramework: net8.0)
Frontend Technology / Framework	HTML5, CSS3, Vanilla JavaScript ES6+; no frontend framework
Backend Technology / Framework	ASP.NET Core 8 Web API using C# controllers, services and repositories
Database used	Microsoft SQL Server LocalDB - RecruitmentProjectDb
Programming Languages	C#, JavaScript; HTML5/CSS3 for UI; JSON configuration; SQL Server via EF Core
APIs / Services used	Internal REST-style JSON APIs, JWT authentication, Swagger/OpenAPI, deterministic job matching, notifications, employer verification, applications/interviews, moderation/reporting services
Other Tools / Libraries	EF Core 8.0.8, SQL Server provider, BCrypt.Net-Next 4.0.3, JwtBearer 8.0.8, Swashbuckle 6.6.2, Microsoft.OpenApi 1.6.22, MemoryCache, Git, EF migrations, ASP.NET Core static file hosting

Overall classification

Smart Recruit is a full-stack .NET web application: a vanilla HTML/CSS/JavaScript multi-page frontend hosted by ASP.NET Core, a C# REST API backend, and a SQL Server relational database managed through Entity Framework Core.

Appendix A - Repository Evidence Reviewed

This document was prepared from the supplied Smart-Team-Project-Test.zip archive. Key evidence files reviewed include:

- Recruitment-Project.csproj - target framework and NuGet package versions.
- Program.cs - database provider, JWT authentication, DI registrations, Swagger, middleware, static-file hosting and routing.
- appsettings.json - SQL Server LocalDB connection, JWT configuration structure and cache configuration.
- Data/AppDbContext.cs and Migrations/ - EF Core entity sets, relationships and schema migrations.
- Controllers/ - REST API route structure for authentication, jobs, employers, applications, interviews, notifications and moderation.
- Services/ and Interfaces/Services/ - business-service implementation and contracts.
- Repositories/ and Interfaces/Repositories/ - data-access layer.
- wwwroot/ and wwwroot/shared/ - frontend technologies, API client, authentication state and role guards.
- Properties/launchSettings.json - local HTTP/HTTPS/IIS Express development profiles.
- .git/ - Git source-control metadata.

END OF DOCUMENT